GenSpark

Loops: while, for, break, and continue

In the last two articles you cleaned input and made decisions about it. Both stopped after one try. If the user typed letters, your program printed a message and ended. A loop fixes that by running a block of code again and again. With loops, your program can keep asking until the answer is valid, count attempts, and stop at the right moment. This article finishes the three skills the Number Guessing Game is built on: checking input, making decisions, and repeating.

Why We Need Loops

Without a loop, repeating an action means copying code:

python

```
print("Hello")
print("Hello")
print("Hello")
```

That works for three lines. It does not work for 100 lines, or for "repeat until the user types a valid answer", because you do not know in advance how many tries that will take. Python has two kinds of loops for this: `while` and `for`.

The while Loop

A `while` loop repeats its indented block as long as its condition is `True`. Before each pass, Python checks the condition. When it becomes `False`, the loop ends and the program continues with the next unindented line.

python

```
count = 3

while count > 0:
    print(count)
    count -= 1

print("Liftoff")
```

Output:

```
3
2
1
Liftoff
```

A `while` loop has the same shape as an `if`: a condition, a colon, and an indented block. The difference is that an `if` runs once and a `while` goes back to check the condition again.

Tracing a loop

When a loop confuses you, trace it on paper. Write down each variable after every pass:

Pass	`count` at the start	Condition `count > 0`	Prints	`count` after
1	3	True	3	2
2	2	True	2	1
3	1	True	1	0
4	0	False	loop ends	0

If the condition is `False` the very first time, the body never runs:

python

```
count = 0
```

```
while count > 0:
    print("This never prints")
```

The three parts of a counting loop

Most `while` loops that count have three parts:

1. **Set up** a variable before the loop: `count = 3`
2. **Check** it in the condition: `while count > 0`
3. **Update** it inside the loop: `count -= 1`

Forget the third part and you get an infinite loop.

Infinite Loops

If the condition never becomes `False`, the loop never ends and your program appears frozen:

python

```
count = 3
while count > 0:
    print(count)      # count never changes, so this prints 3 forever
```

To stop a program that is stuck, click in the Terminal and press `Ctrl+C`. Python stops and shows a `KeyboardInterrupt` message. That is normal and does no harm.

When a loop will not stop, check the three parts. Does the loop change the variable that the condition checks? Does it change it in the right direction?

Counting Attempts

The Number Guessing Game gives the player a limited number of tries. That is a counter loop:

python

```
max_attempts = 3
attempts_used = 0
```

```
while attempts_used < max_attempts:
    answer = input("Say the magic word: ").strip().lower()
    attempts_used += 1

    if answer == "please":
        print("Correct!")
    else:
        print(f"Wrong. You have used {attempts_used} of {max_attempts} attempt
```

This loop ends after 3 tries. But it has a flaw: even after a correct answer, it keeps asking. We need a way to leave early, which is `break`.

break: Leave the Loop

`break` ends the loop immediately, even if the condition is still `True`. Python jumps to the first line after the loop.

python

```
max_attempts = 3
attempts_used = 0

while attempts_used < max_attempts:
    answer = input("Say the magic word: ").strip().lower()
    attempts_used += 1

    if answer == "please":
        print("Correct!")
        break

    print(f"Wrong. You have used {attempts_used} of {max_attempts} attempts.")

print("Done")
```

When the player types `please`, the program prints "Correct!", hits `break`, skips the "Wrong" line, and runs `print("Done")`.

Knowing Why the Loop Ended

A loop can end for more than one reason: the player won, or the attempts ran out. Your code after the loop often needs to know which one happened. The simplest tool is a **flag**, a boolean variable that you set to record what happened:

python

```
max_attempts = 3
attempts_used = 0
won = False                        # assume the player has not won

while attempts_used < max_attempts:
    answer = input("Say the magic word: ").strip().lower()
    attempts_used += 1

    if answer == "please":
        won = True                 # record the win
        break

if won:
    print(f"You got it in {attempts_used} attempts.")
else:
    print("Out of attempts.")
```

After the loop, `won` tells you what happened. This pattern is exactly what the game uses to decide between a win message and a loss message that reveals the number.

continue: Skip to the Next Pass

`continue` skips the rest of the current pass and goes back to the top of the loop. Where `break` leaves the loop, `continue` stays in it and starts over.

This is how the game handles bad input without counting it as an attempt:

python

```
max_attempts = 3
attempts_used = 0
```

```
while attempts_used < max_attempts:
    answer = input("Enter a whole number: ").strip()

    if not answer.isdigit():
        print("That is not a whole number. Try again.")
        continue                # skip the counter and ask again

    attempts_used += 1
    print(f"Attempt {attempts_used} of {max_attempts}: you entered {answer}.")
```

When the player types letters, `continue` jumps back to the top before `attempts_used += 1` runs. Bad input costs nothing. Only valid input is counted.

The continue trap

`continue` skips everything below it, including the line that updates your counter. If the condition depends on a variable that `continue` skips, you get an infinite loop:

python

```
count = 0
while count < 5:
    if count == 2:
        continue          # count stays 2 forever
    count += 1
```

Fix it by updating the variable before you `continue`:

python

```
count = 0
while count < 5:
    count += 1
    if count == 2:
        continue          # now count has already changed
    print(count)          # prints 1, 3, 4, 5
```

In the game, there is no trap. The condition checks `attempts_used`, and the loop skips only the lines that change it, on purpose. Nothing is stuck, because the next pass asks the player for new input.

while True: A Loop That Runs Until break

Sometimes you do not have a counter. You just want to repeat until something is right. The common pattern is `while True`, a loop whose condition is always true, with a `break` inside it for the exit:

python

```
while True:
    choice = input("Pick a level (1 Easy, 2 Medium, 3 Hard): ").strip()

    if choice in ["1", "2", "3"]:
        break                    # valid answer, leave the loop

    print("That is not a valid level. Try again.")

print(f"You picked level {choice}.")
```

This is the menu pattern. The program keeps asking until the answer is valid, and then `break` is the only way out. It is the fix for the problem you saw in the decisions article, where a bad menu answer left the program without any settings.

Use `while True` when you cannot say in advance how many tries it will take. Always make sure there is a `break` that can be reached, or the loop never ends.

After the loop, use the decisions from the last article to turn the choice into settings:

python

```
if choice == "1":
    high = 50
    max_attempts = 10
elif choice == "2":
    high = 100
    max_attempts = 8
else:
    high = 500
```

```
    max_attempts = 10
```

Because the loop already guaranteed a valid choice, the last `else` can handle `"3"`.

The for Loop

A `for` loop goes through a sequence of values, one at a time, and runs its block for each one. Use it when you know how many times to repeat, or when you want to visit every item in something.

python

```
for number in range(5):
    print(number)
```

Output:

```
0
1
2
3
4
```

On each pass, the variable after `for` (here, `number`) takes the next value. You choose the variable name.

range()

`range()` makes a sequence of whole numbers. It has three forms:

Form	Produces	Example	Values
`range(stop)`	0 up to, but not including, stop	`range(4)`	0, 1, 2, 3
`range(start, stop)`	start up to, but not including, stop	`range(1, 5)`	1, 2, 3, 4

`range(start, stop, step)`	counts by step	`range(0, 10, 3)`	0, 3, 6, 9

The stop number is never included. This surprises almost every beginner. To count from 1 to 5, write `range(1, 6)`.

A negative step counts backward:

python

```
for number in range(5, 0, -1):
    print(number)            # 5, 4, 3, 2, 1
```

Looping over text and lists

A `for` loop can go through the characters of a string:

python

```
for letter in "cat":
    print(letter)            # c, then a, then t
```

It can also go through a list, which you will study in Week 2. For now, think of a list as several values in square brackets:

python

```
levels = ["Easy", "Medium", "Hard"]

for level in levels:
    print(f"Level: {level}")
```

Adding things up

A common pattern is an accumulator: a variable that starts at a base value and grows with each pass.

python

```
total = 0
for number in range(1, 6):
    total += number
print(total)                # 15
```

break and continue in a for loop

Both keywords work in a `for` loop the same way as in a `while` loop.

python

```
# break: stop at the first match
for number in range(1, 101):
    if number % 5 == 0 and number % 7 == 0:
        print(f"First number divisible by 5 and 7: {number}")
        break               # prints 35 and stops

# continue: skip some values
for number in range(1, 8):
    if number % 2 == 0:
        continue            # skip even numbers
    print(number)           # prints 1, 3, 5, 7
```

In a `for` loop, `continue` is safe from the infinite loop trap, because the loop moves to the next value on its own.

Choosing Between while and for

Use `for` when	Use `while` when
You know how many times to repeat	You do not know how many times
You are visiting every item in a sequence	You are waiting for something to become true
Example: print a times table	Example: ask until the input is valid

The game uses both: `while True` for the menus, and a `while` with a counter for the guessing. A `for` loop over `range()` is a poor fit for the guessing game. It moves to the next attempt on its own,

so even with `continue`, a bad entry would use up an attempt.

Loops Inside Loops

You can place a loop inside another loop. The inner loop runs completely for every pass of the outer loop. This is how you work through grids, such as the Tic-Tac-Toe board in Week 2.

python

```
for row in range(3):
    for col in range(3):
        print(f"({row},{col})", end=" ")
    print()                  # start a new line after each row
```

Output:

```
(0,0) (0,1) (0,2)
(1,0) (1,1) (1,2)
(2,0) (2,1) (2,2)
```

The outer loop picks a row, and the inner loop visits each column in it. Remember that `break` and `continue` affect only the innermost loop they are in.

Putting It Together

This program is the heart of the Number Guessing Game. It uses every idea in the last three articles: cleaning and checking input, making decisions, and repeating.

python

```
secret_number = 42
low = 1
high = 100
max_attempts = 8
attempts_used = 0
won = False

while attempts_used < max_attempts:
```

```
        guess_text = input(f"Guess a number from {low} to {high}: ").strip()

        # Bad input does not count as an attempt
        if not guess_text.isdigit():
            print("Please type a whole number.")
            continue

        guess = int(guess_text)

        if guess < low or guess > high:
            print(f"Please pick a number from {low} to {high}.")
            continue

        # Only valid guesses reach this line
        attempts_used += 1

        if guess < secret_number:
            print("Too low")
        elif guess > secret_number:
            print("Too high")
        else:
            won = True
            break

        print(f"Attempts remaining: {max_attempts - attempts_used}")

    if won:
        print(f"Correct! You needed {attempts_used} attempts.")
    else:
        print(f"Out of attempts. The number was {secret_number}.")
```

Run it and try:

- a wrong guess, then the right one
- letters, a blank entry, and `0`, and check that "Attempts remaining" does not go down
- eight wrong guesses in a row

Read it slowly and trace the variables. Notice where `continue` skips the counter, where `break` leaves early, and how the `won` flag decides the final message. The secret number is fixed here so you

can predict the results. Making it random, and adding a timer, are the next article's job.

Common Mistakes

Mistake	What happens	Fix
Forgetting to update the loop variable	Infinite loop	Change the variable inside the loop. Stop with `Ctrl+C`.
Updating it in the wrong direction	Infinite loop	Check that `count -= 1` matches `while count > 0`
`continue` skipping the counter update	Infinite loop	Update the counter before `continue`
`while True` with no reachable `break`	Infinite loop	Make sure some path leads to `break`
Off by one with `range()`	One value too few	The stop value is excluded. Use `range(1, 6)` for 1 to 5.
Wrong indentation	Lines run once or on every pass by mistake	Indent exactly the lines that should repeat
Forgetting the colon after `while` or `for`	`SyntaxError`	End the line with `:`
Checking the win after the loop without a flag	Cannot tell win from loss	Set a variable such as `won` before `break`
Counting a bad guess as an attempt	Players lose tries for typos	Use `continue` before the counter line
Changing the list you loop over	Skipped or repeated items	Loop over the list and build a separate result

Try It Yourself

Create a file called `loops.py` and run it with `python3 loops.py`. Try these:

1. Use a `for` loop to print the numbers 1 to 10.

2. Use a `for` loop with `range()` to print the 5 times table from `5 x 1 = 5` through `5 x 10 = 50`.
3. Use a `while` loop to count down from 10 to 1, then print "Liftoff".
4. Ask the user for a password with `while True`. Keep asking until they type the correct one, then print "Welcome".
5. Give the user 3 tries for the password in exercise 4. Use a counter and a `won` flag. After the loop, print a success message or a "locked out" message.
6. Ask for whole numbers until the user types `done`. Reject anything that is not a number or `done` without crashing. When they finish, print how many numbers they entered and the total.
7. Use two `for` loops to print a 4 by 4 grid of the character `#`.

Test each with good and bad input. Try holding down Enter on a blank prompt, and try typing nonsense.

Why This Matters for an FDE

Loops are everywhere in client work: reading every row of a file, retrying a failed request, polling a service until a job finishes, and processing a queue of records. The mistakes are also well known. An infinite loop can hang a production system, and an off by one error can silently skip the last record. Learning to trace a loop, find its exit, and test its edges is a habit that prevents expensive bugs.

Comprehension Check

1. What are the three parts of a counting `while` loop? What goes wrong if you leave out the update?
2. What is the difference between `break` and `continue`?
3. In the guessing loop, why does a guess of `"abc"` not use up an attempt? Which keyword makes that happen?
4. What numbers does `range(2, 7)` produce? What about `range(10, 0, -3)`?
5. Why does the guessing program set `won = False` before the loop? What would you not be able to do after the loop without it?
6. When would you choose `while True` with a `break` instead of a counter?

7. Look at this code. Why does it never finish, and how do you fix it?

python

```
count = 0
while count < 5:
    if count == 3:
        continue
    count += 1
```

← Previous: Control Flow: Decisions with if, elif, and else

Next: Using random and time, and Reading Error Messages →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!